

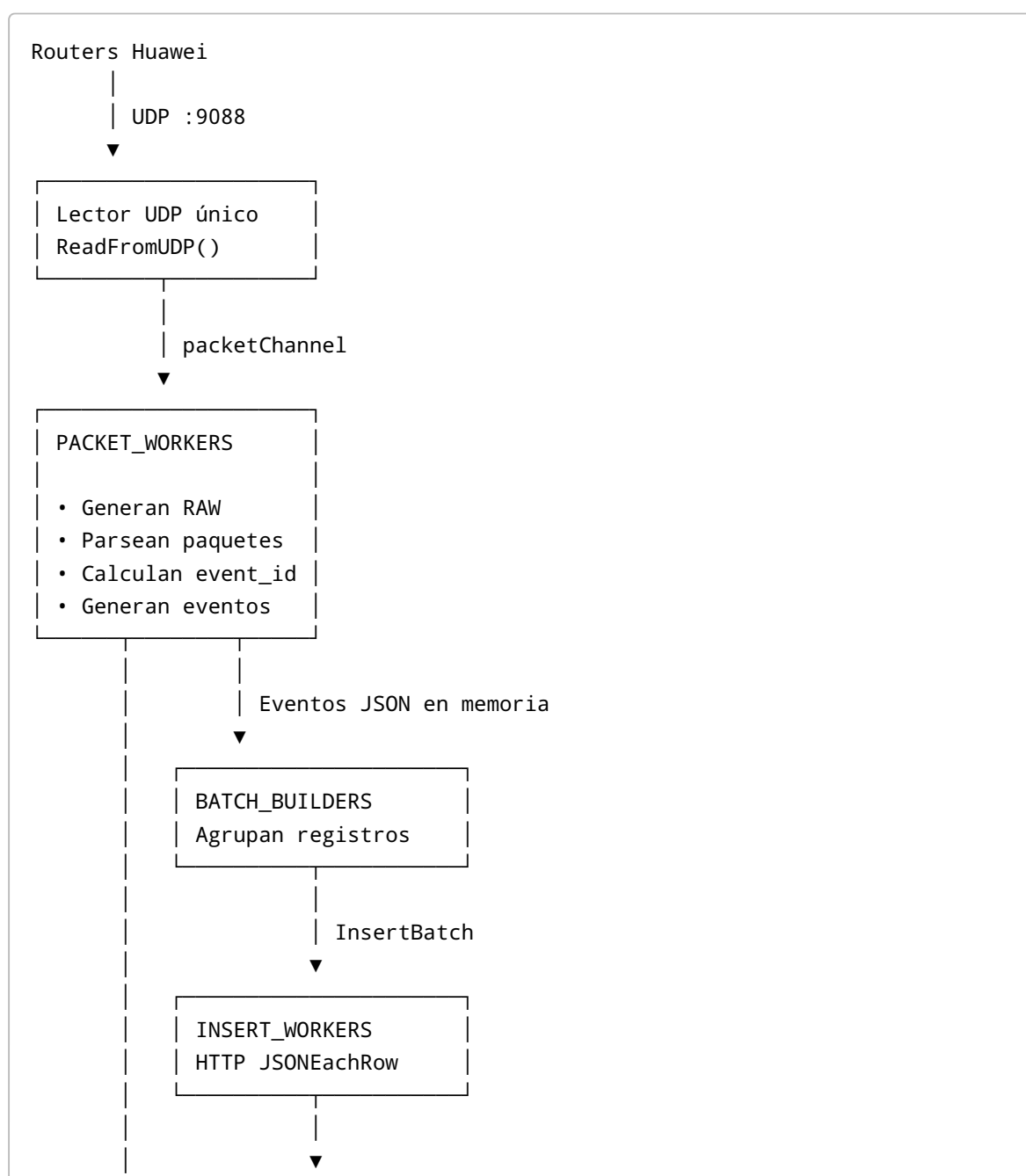
Nuevo flujo del collector Huawei CGN NAT

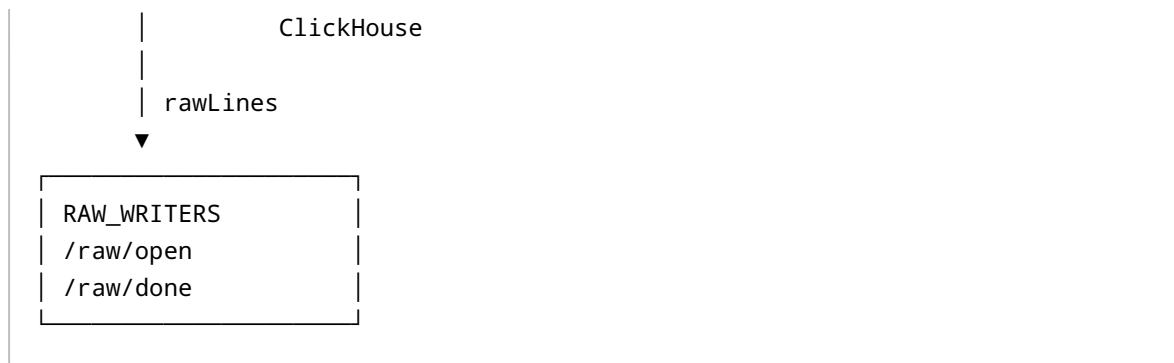
Objetivo

El collector fue rediseñado para soportar un volumen aproximado de hasta **300,000 registros NAT por segundo**, evitando que el procesamiento, la escritura en disco o la inserción en ClickHouse bloqueen la recepción UDP.

El cambio principal consiste en desacoplar la recepción de paquetes del parseo y de la inserción.

Arquitectura general





1. Recepción UDP

El collector escucha en:

0.0.0.0:9088

El goroutine principal únicamente realiza las siguientes operaciones:

1. Recibe el datagrama mediante `ReadFromUDP()`.
2. Copia el contenido a un buffer reutilizable.
3. Registra la IP, puerto y hora de recepción.
4. Coloca el paquete en `packetChannel`.
5. Regresa inmediatamente a `ReadFromUDP()`.

El hilo receptor ya no ejecuta directamente:

- Parseo.
- Conversión hexadecimal.
- Generación de JSON.
- Cálculo de SHA1.
- Escritura en disco.
- Inserción en ClickHouse.

Esto permite vaciar el socket UDP con mayor rapidez y reducir el riesgo de `receive buffer errors`.

Si `packetChannel` se llena, el paquete se descarta y se incrementa:

total_packet_queue_drops

2. Procesamiento paralelo

Varios `PACKET_WORKERS` consumen paquetes desde `packetChannel`.

La cantidad predeterminada es:

CPU lógicos - 2

También puede definirse manualmente mediante:

PACKET_WORKERS

Cada worker realiza dos flujos paralelos sobre el mismo paquete.

Flujo RAW

El paquete recibido se transforma en una línea con:

```
{
  "received_time": "...",
  "router_ip": "...",
  "router_port": 514,
  "packet_size": 208,
  "raw_hex": "..."
}
```

Esta línea se envía a `rawLines`.

Flujo parseado

El worker identifica la estructura del paquete:

16 bytes de cabecera general + N registros de 64 bytes

Ejemplos:

```
80 bytes = 1 registro
144 bytes = 2 registros
208 bytes = 3 registros
272 bytes = 4 registros
```

Por cada registro interno se obtienen:

- `event_time`
- `start_time`
- `end_time`
- `private_ip`
- `private_port`
- `public_ip`
- `public_port`
- `destination_ip`

- `destination_port`
- `event_id`
- `header`
- `packet_size`

La regla aplicada es:

```
event_time = end_time
```

Si el timestamp interno está vacío o es cero, se utiliza como respaldo:

```
timestamp general del paquete
→ hora de recepción del servidor
```

De esta manera se evita guardar fechas equivalentes al Unix epoch, como:

```
1969-12-31 19:00:00
```

3. Persistencia RAW

Los `RAW_WRITERS` consumen las líneas de `rawLines` y las escriben en:

```
/index2/huawei-cgn-go/raw/open
```

Después de alcanzar el límite de cantidad o tiempo, el archivo rota hacia:

```
/index2/huawei-cgn-go/raw/done
```

Aunque internamente hablamos de “RAW” o “binarios”, el formato almacenado continúa siendo:

```
JSONL con el paquete completo codificado en raw_hex
```

Esto mantiene compatibilidad con `reprocess_raw.go`.

El RAW es la copia recuperable de cada paquete UDP.

4. Eliminación del spool JSON parseado

La nueva versión ya no escribe eventos NAT parseados en:

```
/index2/huawei-cgn-go/json
```

Los registros parseados se serializan como JSON únicamente en memoria para enviarlos mediante `JSONEachRow`.

Por lo tanto, el flujo anterior:

```
parseo
→ json/open
→ json/ready
→ json/processing
→ json/done
→ ClickHouse
```

queda eliminado.

El nuevo flujo es:

```
parseo
→ memoria
→ lote
→ ClickHouse
```

Esto reduce:

- Escritura en disco.
- Operaciones de filesystem.
- Cantidad de archivos.
- Uso de espacio.
- Latencia entre parseo e inserción.

5. Construcción de lotes

Los `BATCH_BUILDERS` consumen los eventos desde `eventLines`.

Los registros se agrupan hasta que se cumpla una de estas condiciones:

```
INSERT_BATCH_ROWS
INSERT_BATCH_BYTES
INSERT_FLUSH_MS
```

Configuración inicial:

```
100,000 registros
16 MB
500 milisegundos
```

Cuando se cumple una condición, se genera un `InsertBatch` y se coloca en la cola de inserción.

6. Inserción en ClickHouse

Los `INSERT_WORKERS` consumen lotes y los envían por HTTP mediante:

```
INSERT INTO cgnat.huawei_cgn_nat_v2
FORMAT JSONEachRow
```

Las conexiones HTTP utilizan `keep-alive`.

Si una inserción falla, se realizan reintentos según:

```
INSERT_MAX_RETRIES
INSERT_RETRY_MS
```

Si todos los reintentos fallan:

```
total_insert_errors aumenta
total_insert_dropped_rows aumenta
```

Esas filas dejan de estar disponibles en el flujo en vivo, pero el paquete RAW permanece en disco y puede recuperarse mediante `reprocess_raw.go`.

7. Métricas

Cada 10 segundos el collector registra:

```
rate_received_10s
rate_packet_processed_10s
rate_raw_spooled_10s
rate_parsed_10s
rate_inserted_10s
```

También muestra la ocupación de cada cola:

```
queue_packet
queue_raw
queue_event
queue_batch
```

Interpretación:

```
queue_packet creciendo
→ faltan PACKET_WORKERS

queue_raw creciendo
→ escritura RAW insuficiente

queue_event creciendo
→ batch builders insuficientes

queue_batch creciendo
→ ClickHouse o INSERT_WORKERS insuficientes
```

Contadores críticos:

```
total_packet_queue_drops
total_parse_errors
total_raw_marshal_errors
total_event_marshal_errors
total_insert_errors
total_insert_dropped_rows
```

8. Condición para validar 300,000 registros por segundo

Como las métricas se imprimen cada 10 segundos, el objetivo es:

```
rate_parsed_10s    >= 3,000,000
rate_inserted_10s  >= 3,000,000
```

Además, durante la prueba deben mantenerse en cero:

```
total_packet_queue_drops
total_insert_dropped_rows
UdpRcvbufErrors
```

Y las colas no deben permanecer llenas de manera sostenida.

Diferencia entre el flujo anterior y el nuevo

Flujo anterior

```
ReadFromUDP
→ crear RAW
→ parsear
→ crear JSON
→ escribir JSON en disco
```

- volver a ReadFromUDP
- insertar archivos en ClickHouse

El mismo hilo realizaba demasiado trabajo antes de volver a leer el socket.

Flujo nuevo

```
ReadFromUDP
→ packetChannel
→ volver inmediatamente a ReadFromUDP

PACKET_WORKERS
→ RAW en disco
→ parseo paralelo
→ eventos en memoria

BATCH_BUILDERS
→ lotes

INSERT_WORKERS
→ ClickHouse
```

Resultado esperado

El nuevo diseño busca:

- Aprovechar más CPU mediante procesamiento paralelo.
- Mantener la recepción UDP independiente del parseo.
- Reducir pérdidas por desbordamiento del socket.
- Eliminar el spool JSON parseado.
- Conservar únicamente el RAW recuperable.
- Insertar directamente en ClickHouse.
- Mantener capacidad de reproceso ante fallos.
- Acercarse o superar los 300,000 registros NAT por segundo.